Assignment 7

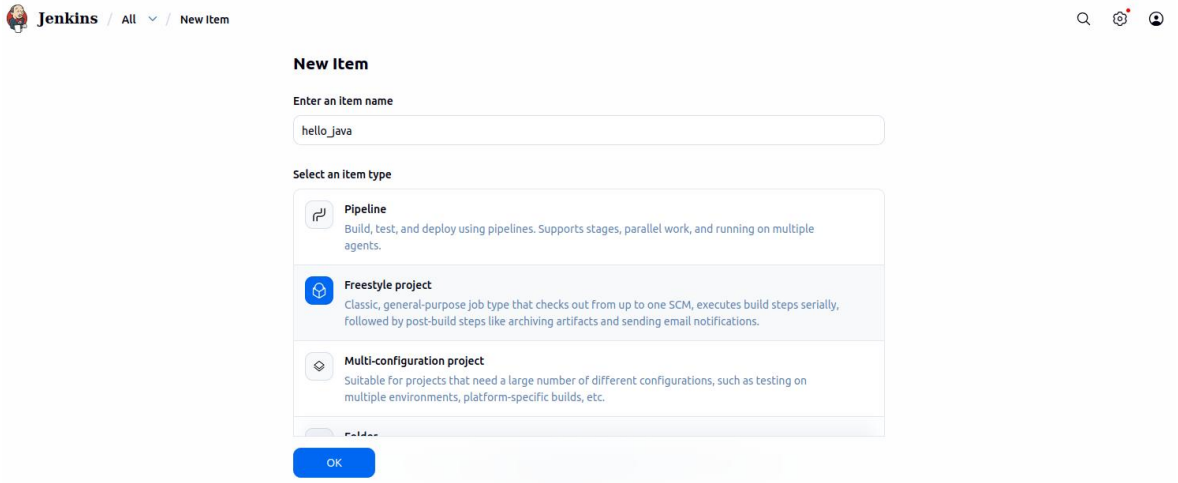
1. Java Application with Jenkins Pipeline

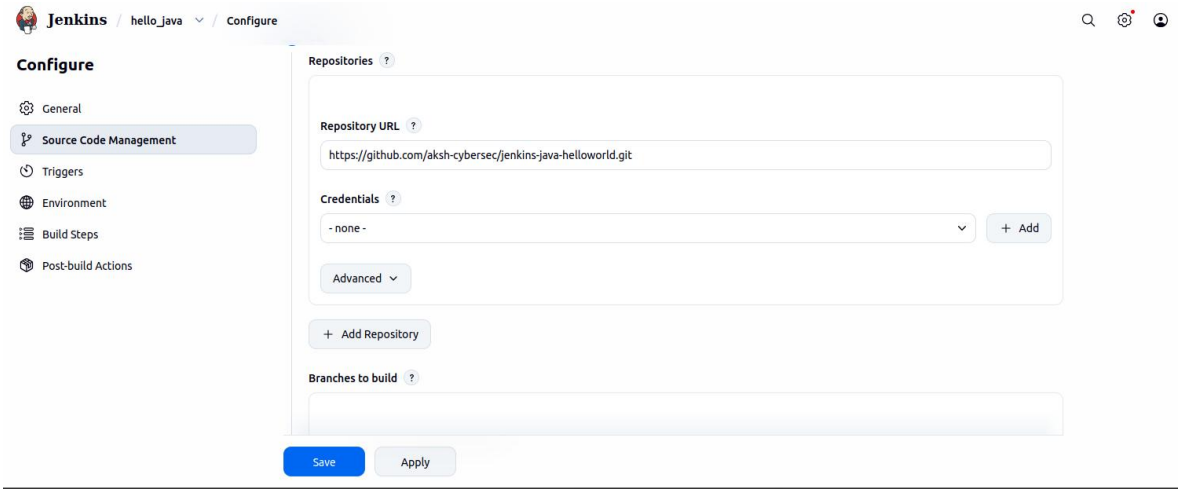
- Create a simple **Java "Hello World" application**.
- Push the source code to a **GitHub repository**.
- Create a **Jenkins Pipeline job** to:
  - Pull the code from GitHub
  - Compile the Java program
  - Execute the application
- Verify the output in Jenkins console.

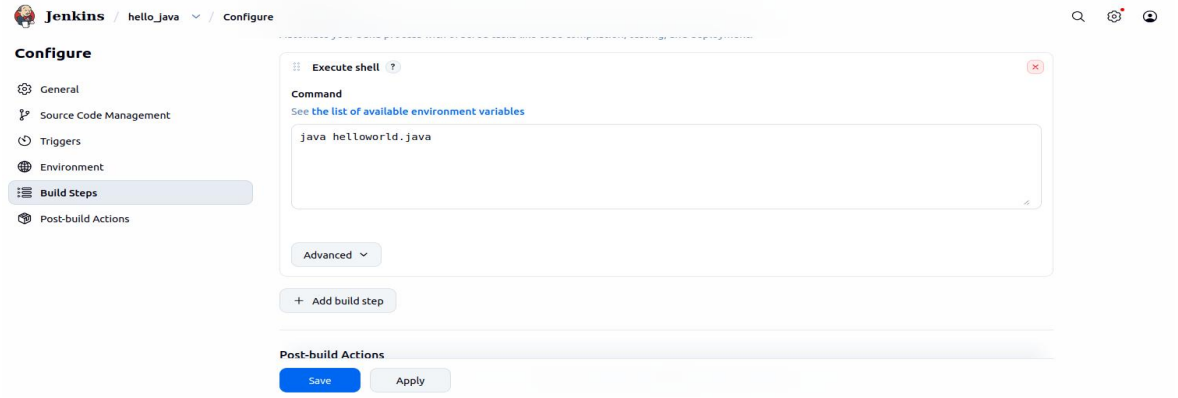
```
server@server-virtual-machine:~/Assignment7$ mkdir hello && cd hello
server@server-virtual-machine:~/Assignment7/hello$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/server/Assignment7/hello/.git/
server@server-virtual-machine:~/Assignment7/hello$ nano helloworld.java
```

```
server@server-virtual-machine: ~/Assignment...
  GNU nano 6.2                  helloworld.java *
class hello{
        public static void main(String[] args){
                System.out.println("Hello World");
        }
}
```

```
server@server-virtual-machine:~/Assignment7/hello$ git add .
server@server-virtual-machine:~/Assignment7/hello$ git commit -m "Added the java file"
[master (root-commit) 4fa6018] Added the java file
 1 file changed, 5 insertions(+)
 create mode 100644 helloworld.java
server@server-virtual-machine:~/Assignment7/hello$ git branch -M main
server@server-virtual-machine:~/Assignment7/hello$ git remote add origin https://github.com/aksh-cybersec/jenkins-java-helloworld.git
server@server-virtual-machine:~/Assignment7/hello$ git push -u origin main
Username for 'https://github.com': aksh-cybersec
Password for 'https://aksh-cybersec@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 316 bytes | 316.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/aksh-cybersec/jenkins-java-helloworld.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
server@server-virtual-machine:~/Assignment7/hello$
```


Jenkins / All / New Item
New Item
Enter an item name
hello_java
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
OK


Jenkins / hello_java / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Repositories
Repository URL
https://github.com/aksh-cybersec/jenkins-java-helloworld.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Save
Apply


Jenkins / hello_java / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Execute shell
Command
See the list of available environment variables
java helloworld.java
Advanced
Add build step
Post-build Actions
Save
Apply


Jenkins / hello_java / #2
Git Build Data
Edit build information
Previous Build
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/hello_java/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/aksh-cybersec/jenkins-java-helloworld.git # timeout=10
Fetching upstream changes from https://github.com/aksh-cybersec/jenkins-java-helloworld.git
> git --version # timeout=10
> git --version # 'git version 2.34.1'
> git fetch --tags --force --progress -- https://github.com/aksh-cybersec/jenkins-java-helloworld.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 4fa60184c02e2c6a404ca3e41230a25bac5602fc (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 4fa60184c02e2c6a404ca3e41230a25bac5602fc # timeout=10
Commit message: "Added the java file"
> git rev-list --no-walk 4fa60184c02e2c6a404ca3e41230a25bac5602fc # timeout=10
[hello_java] $ /bin/sh -xe /tmp/jenkins9605091128057139764.sh
+ java helloworld.java
Hello World
Finished: SUCCESS
Jenkins 2.580.1

2. Docker Image Creation using Jenkins

- Create a **Dockerfile** for the Java application.
- Update the Jenkins Pipeline to:
  - Build a Docker image from the application
  - Run the image as a Docker container locally on the Jenkins server
- Verify the container output.

```
  GNU nano 6.2                                  Dockerfile
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app_java

COPY hello.java /app_java

CMD ["java", "hello.java"]
```

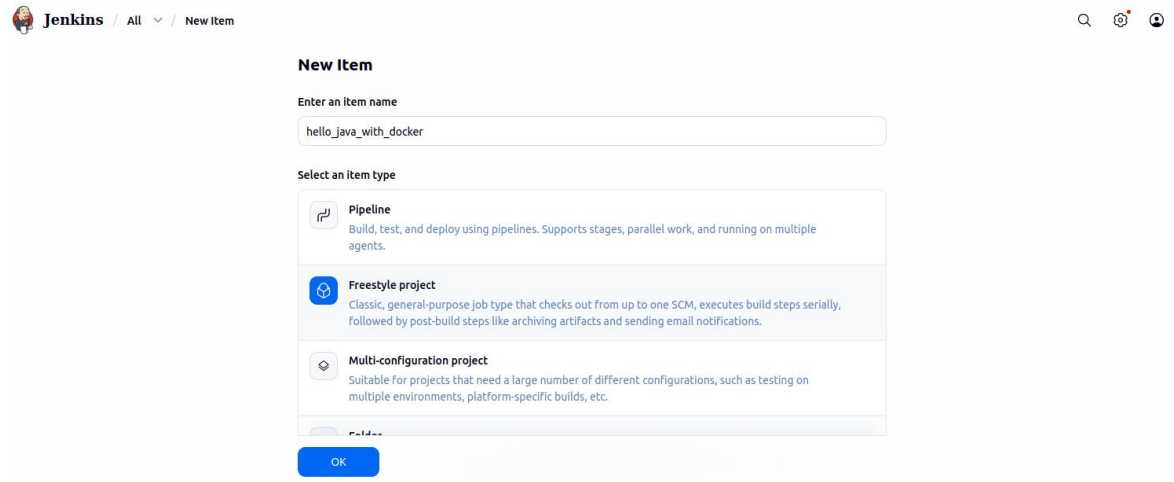
```
server@server-virtual-machine:~/Assignment7/hello$ nano Jenkinfile
server@server-virtual-machine:~/Assignment7/hello$ git add Jenkinfile
server@server-virtual-machine:~/Assignment7/hello$ git commit -m "Added Jenkinfile"
[main 2c174c0] Added Jenkinfile
 1 file changed, 28 insertions(+)
 create mode 100644 Jenkinfile
server@server-virtual-machine:~/Assignment7/hello$ git push origin -u main
Username for 'https://github.com': aksh-cybersec
Password for 'https://aksh-cybersec@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 656 bytes | 656.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/aksh-cybersec/jenkins-java-helloworld.git
   8f4c796..2c174c0  main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

```
  GNU nano 6.2                                                  Jenkinfile
pipeline {
    agent any

    stages {
        stage('Pull Code from GitHub') {
            steps {
                // Jenkins pulls the Dockerfile and Java code from your repo
                git branch: 'main', url: 'https://github.com/aksh-cybersec/jenkins-java-helloworld'
            }
        }

        stage('Build Docker Image') {
            steps {
                // Builds the image using the Dockerfile in the root directory
                sh 'docker build -t java-hello-world:latest .'
            }
        }

        stage('Run Docker Container Locally') {
            steps {
                // Removes any older container with the same name and runs the new one
                sh '
                    docker run --name java-app-container java-hello-world:latest
                '
            }
        }
    }
}
```





3. Push Docker Image to DockerHub

- Configure DockerHub credentials in Jenkins.
- Extend the Jenkins Pipeline to:
  - Tag the Docker image properly
  - Push the image to **DockerHub repository**
- Verify the image is available in DockerHub.

```
 GNU nano 6.2                                                    Jenkinsfile
pipeline {
    agent any

    environment {
        // Your exact DockerHub username
        DOCKER_USER = 'akshadasable'
    }

    stages {
        stage('Pull Code from GitHub') {
            steps {
                // Jenkins pulls the Dockerfile and Java code from your repo
                git branch: 'main', url: 'https://github.com/aksh-cybersec/jenkins-java-helloworld'
            }
        }

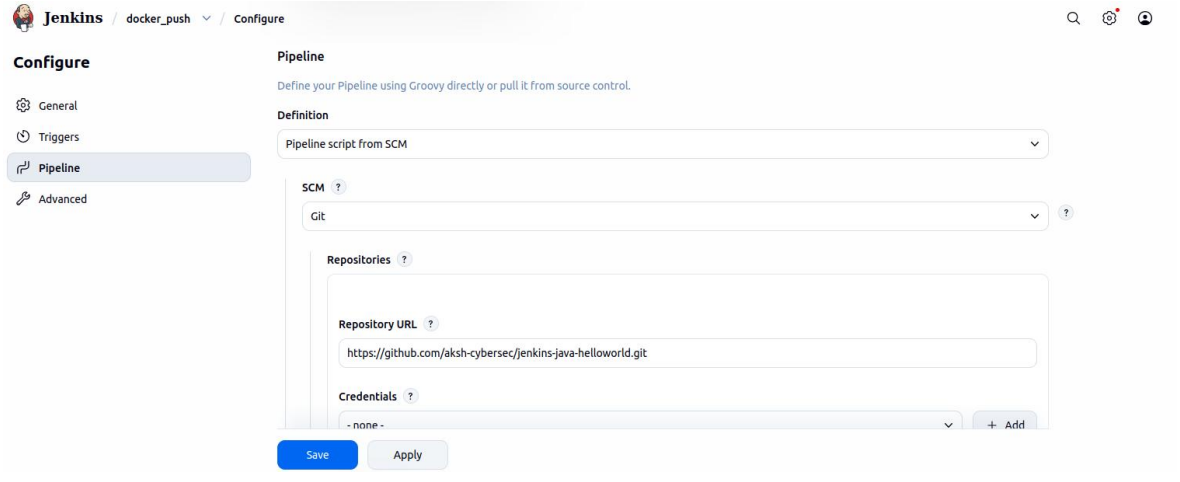
        stage('Build Docker Image') {
            steps {
                // Builds the local image exactly like your previous layout
                sh 'docker build -t java-hello-world:latest .'
            }
        }

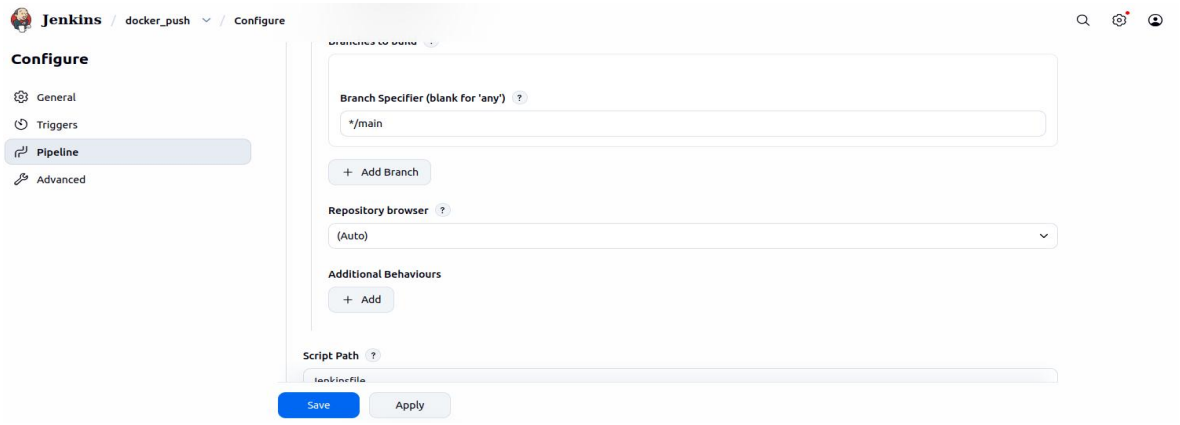
        stage('Run Docker Container Locally') {
            steps {
                // Safely cleans up the older container instance if it exists, then runs it locally
                sh '''
                    docker rm -f java-app-container || true
                    docker run -d --name java-app-container java-hello-world:latest
                '''
            }
        }
```

```
        stage('Tag & Push to DockerHub') {
            steps {
                // Binds your precise Jenkins credentials ID
                withCredentials([usernamePassword(credentialsId: 'my-docker-hub-credentials-id', passwordVariable: 'DOCKER_PASS_ENV', usernameVariable: 'DOCKER_USER_EN

                    // Uses single quotes to prevent Groovy from intercepting the shell variables
                    sh 'echo "$DOCKER_PASS_ENV" | docker login -u "$DOCKER_USER_ENV" --password-stdin'

                    // Properly tags and pushes your local image using your target DockerHub account prefix
                    sh "docker tag java-hello-world:latest ${DOCKER_USER}/java-hello-world:latest"
                    sh "docker push ${DOCKER_USER}/java-hello-world:latest"
                }
            }
        }
    }
}
```







4. Deploy Application to Kubernetes using Jenkins

- Create a **Kubernetes deployment and service YAML file**.
- Extend the Jenkins Pipeline to:

Pull the Docker image from DockerHub

  - Deploy the application as a **Kubernetes Pod**
- Verify the pod is running successfully in the cluster.

```
 GNU nano 6.2                                  deployment.yml *
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app-deployment
  labels:
    app: java-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      containers:
      - name: java-app
        image: akshadasable/java-hello-world:latest
        imagePullPolicy: Always
---
apiVersion: v1
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: java-app-service
spec:
  type: NodePort
  selector:
    app: java-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
      nodePort: 30007
```

```
 GNU nano 6.2                                  Jenkinsfile
pipeline {
    agent any

    environment {
        DOCKER_USER = 'akshadasable'
        IMAGE_NAME = "akshadasable/java-hello-world:latest"
    }

    stages {
        stage('Pull Code from GitHub') {
            steps {
                git branch: 'main', url: 'https://github.com/aksh-cybersec/jenkins-java-helloworld'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME} ."
            }
        }

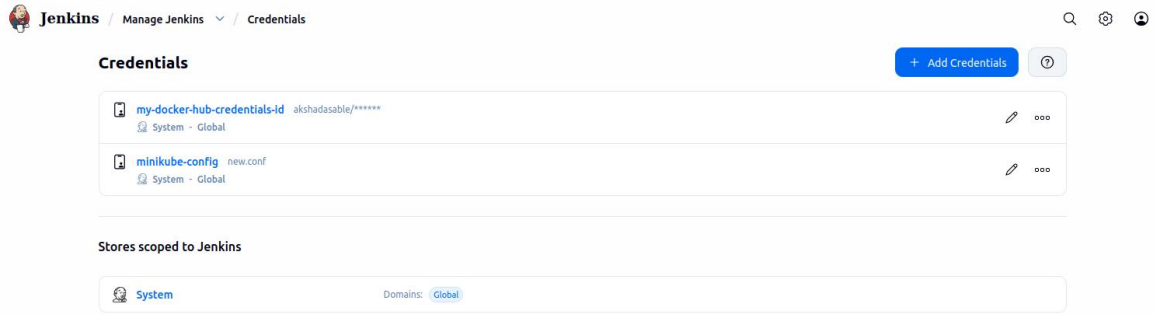
        stage('Run Docker Container Locally') {
            steps {
                sh '''
                    docker rm -f java-app-container || true
                    docker run -d --name java-app-container ${IMAGE_NAME}
                '''
            }
```

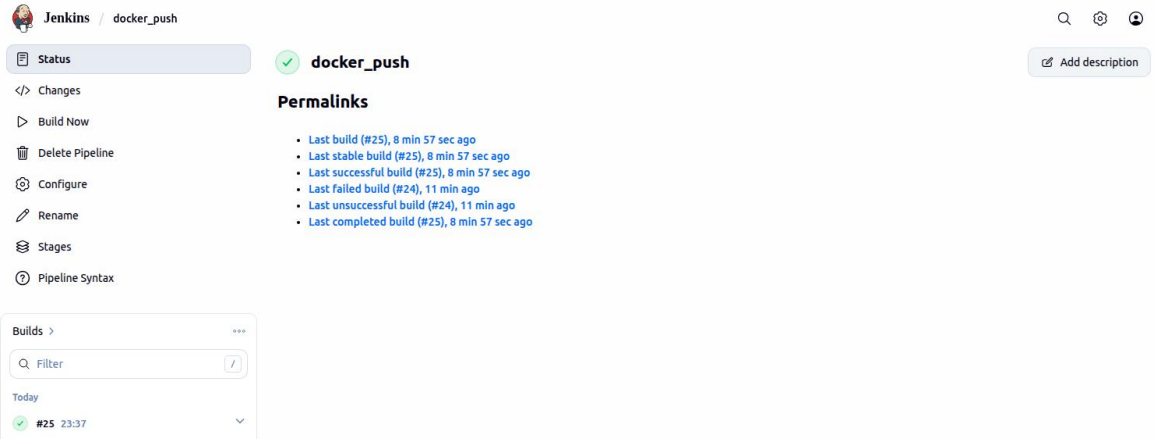
```
 GNU nano 6.2                                  Jenkinsfile *
        stage('Tag & Push to DockerHub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'my-docker-hub-credentials-id',
                        passwordVariable: 'DOCKER_PASS_ENV', usernameVariable: 'DOCKER_USER_ENV')]) {
                    sh '''
                        set +x
                        echo "$DOCKER_PASS_ENV" | docker login -u "$DOCKER_USER_ENV" --password-stdin
                    '''
                    sh "docker push ${IMAGE_NAME}"
                }
            }
        }

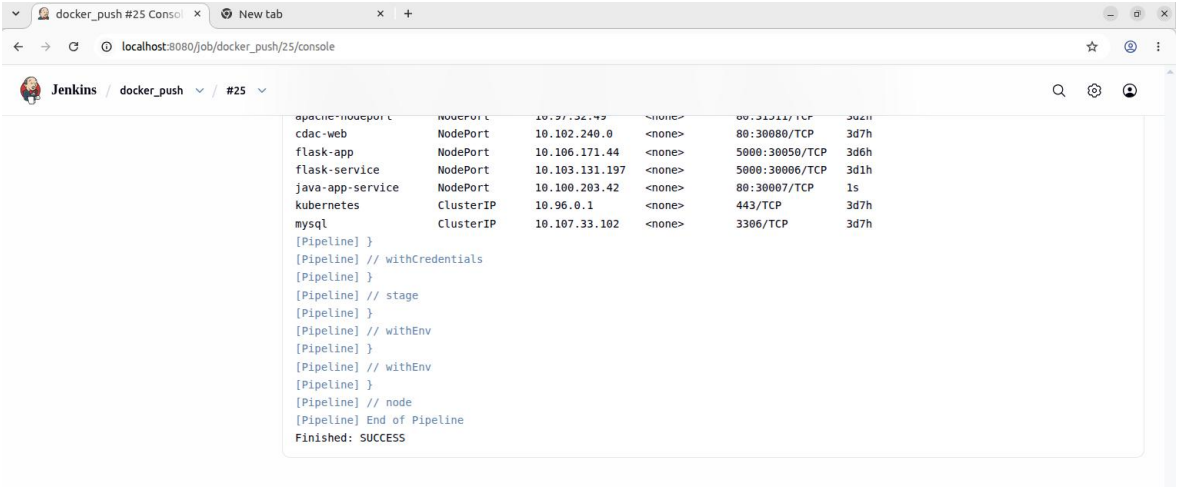
        stage('Debug Kubernetes Connection') {
            steps {
                withCredentials([file(credentialsId: 'minikube-config', variable: 'KUBECONFIG')]) {
                    sh '''
                        set -eu
                        echo "Checking Kubernetes context..."
                        kubectl config current-context
                        echo "Checking cluster nodes..."
                        kubectl get nodes
                    '''
                }
            }
        }
```

```
 GNU nano 6.2                                  Jenkinsfile *
        stage('Deploy to Kubernetes') {
            steps {
                withCredentials([file(credentialsId: 'minikube-config', variable: 'KUBECONFIG')]) {
                    sh '''
                        set -eu
                        kubectl apply -f deployment.yaml
                    '''
                }
            }
        }

        stage('Verify Deployment') {
            steps {
                withCredentials([file(credentialsId: 'minikube-config', variable: 'KUBECONFIG')]) {
                    sh '''
                        set -eu
                        echo "Verifying deployments, pods, and services..."
                        kubectl get deployments
                        kubectl get pods
                        kubectl get services
                    '''
                }
            }
        }
    }
}
```


Jenkins / Manage Jenkins / Credentials
Credentials
+ Add Credentials
my-docker-hub-credentials-id akshadasable/******
System - Global
minikube-config new.conf
System - Global
Stores scoped to Jenkins
System
Domains: Global


Jenkins / docker_push
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
docker_push
Add description
Permalinks
Last build (#25), 8 min 57 sec ago
Last stable build (#25), 8 min 57 sec ago
Last successful build (#25), 8 min 57 sec ago
Last failed build (#24), 11 min ago
Last unsuccessful build (#24), 11 min ago
Last completed build (#25), 8 min 57 sec ago
Builds
Filter
Today
#25 23:37


docker_push #25 Consol
New tab
localhost:8080/job/docker_push/25/console
Jenkins / docker_push / #25
cdac-web NodePort 10.102.240.0 <none> 80:30080/TCP 3d7h
flask-app NodePort 10.106.171.44 <none> 5000:30050/TCP 3d6h
flask-service NodePort 10.103.131.197 <none> 5000:30006/TCP 3d1h
java-app-service NodePort 10.100.203.42 <none> 80:30007/TCP 1s
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 3d7h
mysql ClusterIP 10.107.33.102 <none> 3306/TCP 3d7h
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS